In [66]:
```
# # This mounts your Google Drive to the Colab VM.
# from google.colab import drive
# drive.mount('/content/drive')

# # TODO: Enter the foldername in your Drive where you have saved the unzipped
# # assignment folder, e.g. 'assignment4'
# FOLDERNAME = 'assignment4'
# assert FOLDERNAME is not None, "[!] Enter the foldername."

# # Now that we've mounted your Drive, this ensures that
# # the Python interpreter of the Colab VM can load
# # python files from within it.
# import sys
# sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# # This downloads the CIFAR-10 dataset to your Drive
# # if it doesn't already exist.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs6353/datasets/
# !bash get_datasets.sh
# %cd /content/drive/My\ Drive/$FOLDERNAME
```

In [67]:
```
%cd "/home/jared/CS-6353/Homeworks/assignment4/cs6353/datasets/"
!bash get_datasets.sh
%cd "/home/jared/CS-6353/Homeworks/assignment4/"
```

```
/home/jared/CS-6353/Homeworks/assignment4/cs6353/datasets
4972.72s - pydevd: Sending message related to process being replaced timed-out after
5 seconds
--2025-11-16 13:51:12--  https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
Resolving www.cs.toronto.edu (www.cs.toronto.edu)... 128.100.3.30
Connecting to www.cs.toronto.edu (www.cs.toronto.edu)|128.100.3.30|:443... connected
.
HTTP request sent, awaiting response... 200 OK
Length: 170498071 (163M) [application/x-gzip]
Saving to: ‘cifar-10-python.tar.gz’

cifar-10-python.tar 100%[===================>] 162.60M   371KB/s    in 7m 7s

2025-11-16 13:59:00 (390 KB/s) - ‘cifar-10-python.tar.gz’ saved [170498071/17049807
1]

cifar-10-batches-py/
cifar-10-batches-py/data_batch_4
cifar-10-batches-py/readme.html
cifar-10-batches-py/test_batch
cifar-10-batches-py/data_batch_3
cifar-10-batches-py/batches.meta
cifar-10-batches-py/data_batch_2
cifar-10-batches-py/data_batch_5
cifar-10-batches-py/data_batch_1
/home/jared/CS-6353/Homeworks/assignment4
```

Dropout

Dropout [1] is a technique for regularizing neural networks by randomly setting some features to zero during the forward pass. In this exercise you will implement a dropout layer and modify your fully-connected network to optionally use dropout.

[1] Geoffrey E. Hinton et al, "Improving neural networks by preventing co-adaptation of feature detectors", arXiv 2012

In [68]:
```
# As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from cs6353.classifiers.fc_net import *
from cs6353.data_utils import get_CIFAR10_data
from cs6353.gradient_check import eval_numerical_gradient, eval_numerical_gradient_
from cs6353.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
  """ returns relative error """
  return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

# You can ignore the message that asks you to run Python script for now.
# It will be required in the second part of the assignment.
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

In [69]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k, v in data.items():
  print('%s: ' % k, v.shape)
```

```
X_train:  (49000, 3, 32, 32)
y_train:  (49000,)
X_val:  (1000, 3, 32, 32)
y_val:  (1000,)
X_test:  (1000, 3, 32, 32)
y_test:  (1000,)
```

Dropout forward pass

In the file `cs6353/layers.py` , implement the forward pass for dropout. Since dropout

behaves differently during training and testing, make sure to implement the operation for both modes.

Once you have done so, run the cell below to test your implementation.

In [92]:
```
np.random.seed(231)
x = np.random.randn(500, 500) + 10

for p in [0.75, 0.6, 0.3]:
  out, _ = dropout_forward(x, {'mode': 'train', 'p': p})
  out_test, _ = dropout_forward(x, {'mode': 'test', 'p': p})

  print('Running tests with p = ', p)
  print('Mean of input: ', x.mean())
  print('Mean of train-time output: ', out.mean())
  print('Mean of test-time output: ', out_test.mean())
  print('Fraction of train-time output set to zero: ', (out == 0).mean())
  print('Fraction of test-time output set to zero: ', (out_test == 0).mean())
  print()
```

```
Running tests with p =  0.75
Mean of input:  10.000207878477502
Mean of train-time output:  10.006234670544599
Mean of test-time output:  10.000207878477502
Fraction of train-time output set to zero:  0.749832
Fraction of test-time output set to zero:  0.0

Running tests with p =  0.6
Mean of input:  10.000207878477502
Mean of train-time output:  10.035153558044966
Mean of test-time output:  10.000207878477502
Fraction of train-time output set to zero:  0.598632
Fraction of test-time output set to zero:  0.0

Running tests with p =  0.3
Mean of input:  10.000207878477502
Mean of train-time output:  10.007776657908957
Mean of test-time output:  10.000207878477502
Fraction of train-time output set to zero:  0.299504
Fraction of test-time output set to zero:  0.0
```

Dropout backward pass

In the file `cs6353/layers.py`, implement the backward pass for dropout. After doing so, run the following cell to numerically gradient-check your implementation.

In [93]:
```
np.random.seed(231)
x = np.random.randn(10, 10) + 10
dout = np.random.randn(*x.shape)

dropout_param = {'mode': 'train', 'p': 0.8, 'seed': 123}
out, cache = dropout_forward(x, dropout_param)
```

```
dx = dropout_backward(dout, cache)
dx_num = eval_numerical_gradient_array(lambda xx: dropout_forward(xx, dropout_param

# Error should be around e-10 or less
print('dx relative error: ', rel_error(dx, dx_num))
```

```
dx relative error:  1.8929048652720146e-11
```

Inline Question 1:

What happens if we do not divide the values being passed through inverse dropout by `p` in the dropout layer? Why does that happen?

Answer:

- Response: The first thing I noticed was that the mean of the train time output and the mean of the test time output were no longer close to one another. We remember that dropout only happens in training and not in testing so if we want a consistent amount of signal to pass through we want to scale all of the proabilities that weren't dropped.

Fully-connected nets with Dropout

In the file `cs6353/classifiers/fc_net.py`, modify your implementation to use dropout. Specifically, if the constructor of the net receives a value that is not 0 for the `dropout` parameter, then the net should add dropout immediately after every ReLU nonlinearity. After doing so, run the following to numerically gradient-check your implementation.

In [102…

```
np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for dropout in [0, 0.25, 0.5]:
  print('Running check with dropout = ', dropout)
  model = FullyConnectedNet([H1, H2], input_dim=D, num_classes=C,
                            weight_scale=5e-2, dtype=np.float64,
                            dropout=dropout, seed=123)

  loss, grads = model.loss(X, y)
  print('Initial loss: ', loss)

  # Relative errors should be around e-6 or less; Note that it's fine
  # if for dropout = 0 you have W2 error be on the order of e-5.
  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=False, h=1e-5
    print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
  print()
```

```
Running check with dropout =  0
Initial loss:  2.300479089768492
W1 relative error: 1.03e-07
W2 relative error: 2.21e-05
W3 relative error: 4.56e-07
b1 relative error: 4.66e-09
b2 relative error: 2.09e-09
b3 relative error: 1.69e-10

Running check with dropout =  0.25
Initial loss:  2.2924325088330475
W1 relative error: 1.76e-08
W2 relative error: 2.98e-09
W3 relative error: 4.29e-09
b1 relative error: 7.78e-10
b2 relative error: 3.36e-10
b3 relative error: 1.59e-10

Running check with dropout =  0.5
Initial loss:  2.30427592207859
W1 relative error: 3.11e-07
W2 relative error: 2.48e-08
W3 relative error: 6.43e-08
b1 relative error: 5.37e-09
b2 relative error: 1.91e-09
b3 relative error: 1.85e-10
```

Regularization experiment

As an experiment, we will train a pair of two-layer networks on 500 training examples: one will use no dropout, and one will use a dropout probability of 0.75. We will then visualize the training and validation accuracies of the two networks over time.

In [103…

```
# Train two identical nets, one with dropout and one without
np.random.seed(231)
num_train = 500
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

solvers = {}
dropout_choices = [0, 0.75]
for dropout in dropout_choices:
  model = FullyConnectedNet([500], dropout=dropout)
  print(dropout)

  solver = Solver(model, small_data,
                  num_epochs=25, batch_size=100,
                  update_rule='adam',
```

```
                optim_config={
                  'learning_rate': 5e-4,
                },
                verbose=True, print_every=100)
solver.train()
solvers[dropout] = solver
```

```
0
(Iteration 1 / 125) loss: 7.856642
(Epoch 0 / 25) train acc: 0.260000; val_acc: 0.184000
(Epoch 1 / 25) train acc: 0.416000; val_acc: 0.258000
(Epoch 2 / 25) train acc: 0.482000; val_acc: 0.276000
(Epoch 3 / 25) train acc: 0.532000; val_acc: 0.277000
(Epoch 4 / 25) train acc: 0.600000; val_acc: 0.271000
(Epoch 5 / 25) train acc: 0.708000; val_acc: 0.299000
(Epoch 6 / 25) train acc: 0.722000; val_acc: 0.282000
(Epoch 7 / 25) train acc: 0.832000; val_acc: 0.255000
(Epoch 8 / 25) train acc: 0.878000; val_acc: 0.269000
(Epoch 9 / 25) train acc: 0.902000; val_acc: 0.275000
(Epoch 10 / 25) train acc: 0.890000; val_acc: 0.261000
(Epoch 11 / 25) train acc: 0.930000; val_acc: 0.282000
(Epoch 12 / 25) train acc: 0.958000; val_acc: 0.300000
(Epoch 13 / 25) train acc: 0.964000; val_acc: 0.305000
(Epoch 14 / 25) train acc: 0.962000; val_acc: 0.318000
(Epoch 15 / 25) train acc: 0.966000; val_acc: 0.303000
(Epoch 16 / 25) train acc: 0.982000; val_acc: 0.307000
(Epoch 17 / 25) train acc: 0.968000; val_acc: 0.322000
(Epoch 18 / 25) train acc: 0.992000; val_acc: 0.316000
(Epoch 19 / 25) train acc: 0.984000; val_acc: 0.303000
(Epoch 20 / 25) train acc: 0.980000; val_acc: 0.306000
(Iteration 101 / 125) loss: 0.127809
(Epoch 21 / 25) train acc: 0.982000; val_acc: 0.305000
(Epoch 22 / 25) train acc: 0.954000; val_acc: 0.314000
(Epoch 23 / 25) train acc: 0.954000; val_acc: 0.320000
(Epoch 24 / 25) train acc: 0.980000; val_acc: 0.313000
(Epoch 25 / 25) train acc: 0.960000; val_acc: 0.308000
0.75
(Iteration 1 / 125) loss: 19.352449
(Epoch 0 / 25) train acc: 0.256000; val_acc: 0.193000
(Epoch 1 / 25) train acc: 0.372000; val_acc: 0.245000
(Epoch 2 / 25) train acc: 0.444000; val_acc: 0.276000
(Epoch 3 / 25) train acc: 0.520000; val_acc: 0.271000
(Epoch 4 / 25) train acc: 0.572000; val_acc: 0.313000
(Epoch 5 / 25) train acc: 0.622000; val_acc: 0.309000
(Epoch 6 / 25) train acc: 0.628000; val_acc: 0.297000
(Epoch 7 / 25) train acc: 0.656000; val_acc: 0.308000
(Epoch 8 / 25) train acc: 0.704000; val_acc: 0.314000
(Epoch 9 / 25) train acc: 0.730000; val_acc: 0.319000
(Epoch 10 / 25) train acc: 0.764000; val_acc: 0.312000
(Epoch 11 / 25) train acc: 0.806000; val_acc: 0.311000
(Epoch 12 / 25) train acc: 0.804000; val_acc: 0.296000
(Epoch 13 / 25) train acc: 0.820000; val_acc: 0.295000
(Epoch 14 / 25) train acc: 0.768000; val_acc: 0.299000
(Epoch 15 / 25) train acc: 0.856000; val_acc: 0.307000
(Epoch 16 / 25) train acc: 0.830000; val_acc: 0.317000
(Epoch 17 / 25) train acc: 0.868000; val_acc: 0.315000
(Epoch 18 / 25) train acc: 0.870000; val_acc: 0.319000
(Epoch 19 / 25) train acc: 0.882000; val_acc: 0.333000
(Epoch 20 / 25) train acc: 0.894000; val_acc: 0.322000
(Iteration 101 / 125) loss: 3.982434
(Epoch 21 / 25) train acc: 0.884000; val_acc: 0.286000
(Epoch 22 / 25) train acc: 0.892000; val_acc: 0.306000
(Epoch 23 / 25) train acc: 0.906000; val_acc: 0.303000
```

```
(Epoch 24 / 25) train acc: 0.926000; val_acc: 0.296000
(Epoch 25 / 25) train acc: 0.914000; val_acc: 0.318000
```

In [104…

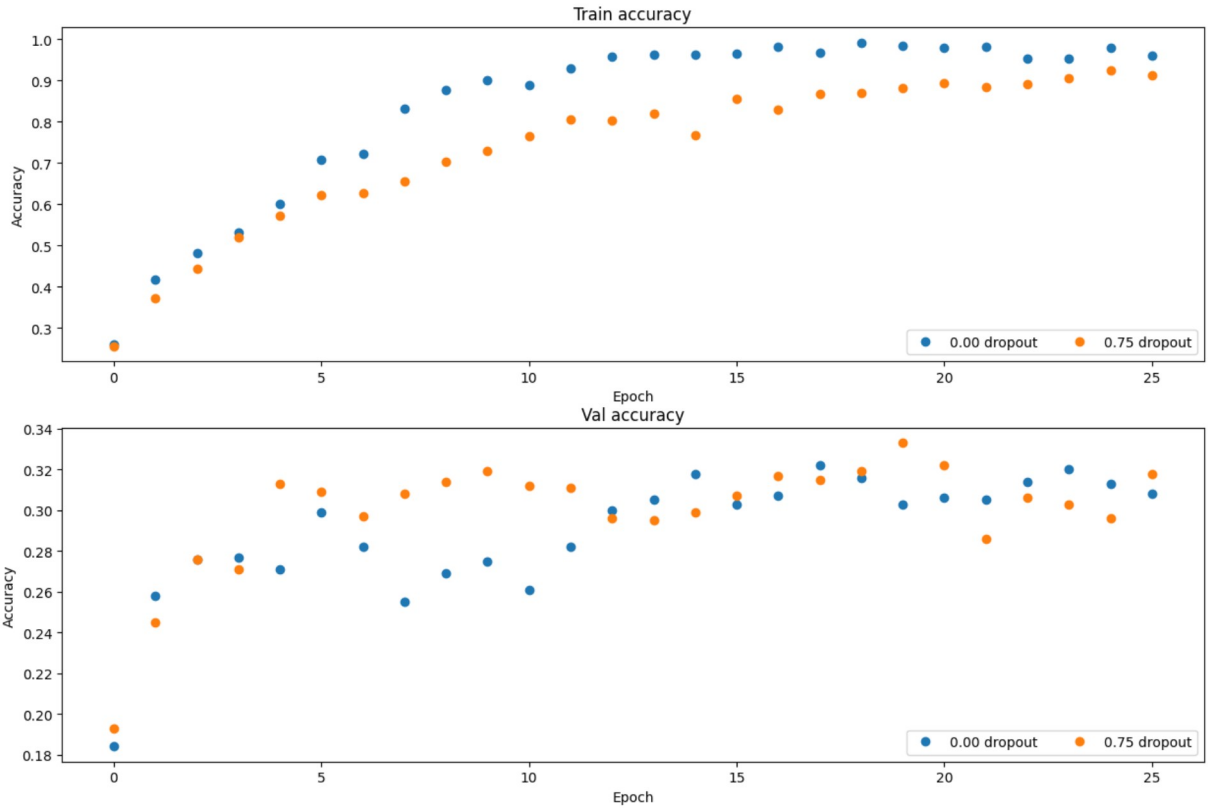
```python
# Plot train and validation accuracies of the two models

train_accs = []
val_accs = []
for dropout in dropout_choices:
  solver = solvers[dropout]
  train_accs.append(solver.train_acc_history[-1])
  val_accs.append(solver.val_acc_history[-1])

plt.subplot(3, 1, 1)
for dropout in dropout_choices:
  plt.plot(solvers[dropout].train_acc_history, 'o', label='%.2f dropout' % dropout)
plt.title('Train accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(ncol=2, loc='lower right')

plt.subplot(3, 1, 2)
for dropout in dropout_choices:
  plt.plot(solvers[dropout].val_acc_history, 'o', label='%.2f dropout' % dropout)
plt.title('Val accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(ncol=2, loc='lower right')

plt.gcf().set_size_inches(15, 15)
plt.show()
```



Inline Question 2:

Compare the validation and training accuracies with and without dropout -- what do your results suggest about dropout as a regularizer?

Answer:

- Response: The graphs show that you can use dropout as a regularizer and it helps but as you continue training you will still start to overfit on your training data. Dropout forces our network to not rely on a set of neurons and introduces redundency into our network. With dropout overfitting was consistently less on the interval of 25 epochs. It was interesting to see how much dropout helped on our validation accuracy when our epochs was low. As we reached 25 iterations the results got closer to one another.

Inline Question 3:

Suppose we are training a deep fully-connected network for image classification, with dropout after hidden layers (parameterized by dropout probability p). How should we modify p, if at all, if we decide to decrease the size of the hidden layers (that is, the number of nodes in each layer)?

Answer:

- Response: Dropout is based off of each individual neuron already. For example each neuron has a probability of being "dropped" independent on the number of values in the hidden layer.
- If you were worried about not having enough neurons after dropout to faciliatate effective learning then you could decrease your probability of dropping a neuron.
- If we had a dropout rate of 75% for a very wide network we still have a lot of neurons that can help us learn. If instead we had that same dropout rate for a very small width network we may lose the representational power we need to learn effectively.
- Because of the reasons mentioned above I think you would normally want to decrease your dropout probability if you decrease the size of your network. This is very problem dependant like most things in the field but it could be considered a "decent" rule of thumb.